Aarya Arban

T11 – 05

Assignment No. 9

Weka (Waikato Environment for Knowledge Analysis) is a popular, open-source data mining and machine learning software developed at the University of Waikato in New Zealand. It is written in Java and provides a collection of visualization tools and algorithms for data analysis and predictive modeling, along with graphical user interfaces for easy access to these functions.

How Weka is Used in Data Mining

Weka is widely used in data mining tasks such as:

1. **Data Preprocessing**
    - Cleaning data, removing duplicates, normalization, etc.
    - Supports different data formats like CSV and ARFF.
2. **Classification**
    - Apply machine learning algorithms like Decision Trees (J48), Naive Bayes, k-NN, SVM, etc.
3. **Clustering**
    - Group similar instances using algorithms like k-Means, DBSCAN, etc.
4. **Association Rule Mining**
    - Discover interesting relations using algorithms like Apriori.
5. **Regression**
    - Perform linear and non-linear regression analysis.
6. **Visualization**
    - Visualize datasets, decision trees, clusters, and evaluation metrics.
7. **Evaluation**

- Cross-validation, confusion matrix, ROC curves, and other performance metrics.

Comparison of Weka and Other Data Mining Tools

---

1. Weka

- **Language**: Java
- **Interface**: GUI + Command Line
- **Ease of Use**: Very beginner-friendly
- **Best For**: Education, prototyping ML models
- **Features**: Classification, clustering, association, regression, visualization
- **Limitation**: Not ideal for big data or deep learning

---

2. RapidMiner

- **Language**: Java
- **Interface**: GUI-based drag-and-drop
- **Ease of Use**: Very user-friendly, no coding required
- **Best For**: Business analytics, fast prototyping
- **Features**: ML, data prep, visualization, deployment
- **Limitation**: Limited free version, commercial license for full features

---

3. KNIME

- **Language**: Java
- **Interface**: Visual workflow GUI
- **Ease of Use**: Easy to learn for non-programmers
- **Best For**: Data integration + machine learning
- **Features**: Data cleaning, analytics, reporting, ML
- **Limitation**: Limited advanced deep learning capabilities out-of-the-box

---

4. Orange

- **Language**: Python
- **Interface**: GUI + Python scripting
- **Ease of Use**: Highly intuitive visual programming
- **Best For**: Education, visual data exploration
- **Features**: Classification, clustering, text mining, visualization
- **Limitation**: Less support for big data or complex pipelines

---

5. R / RStudio

- **Language**: R
- **Interface**: Code-based (IDE)
- **Ease of Use**: Steeper learning curve
- **Best For**: Statistical analysis, academic research
- **Features**: Extensive ML packages, data visualization
- **Limitation**: Needs programming knowledge

---

6. Python (Scikit-learn, Pandas, TensorFlow, etc.)

- **Language**: Python
- **Interface**: Code-based (Jupyter, IDE)
- **Ease of Use**: Great for programmers
- **Best For**: Custom ML workflows, scalability
- **Features**: ML, DL, big data, visualization, automation
- **Limitation**: Requires coding and understanding of ML concepts

---

7. SAS Enterprise Miner

- **Language**: SAS
- **Interface**: GUI
- **Ease of Use**: Designed for business users
- **Best For**: Enterprise-level analytics
- **Features**: Predictive modeling, data prep, scoring
- **Limitation**: Expensive, limited to SAS ecosystem

---

8. IBM SPSS Modeler

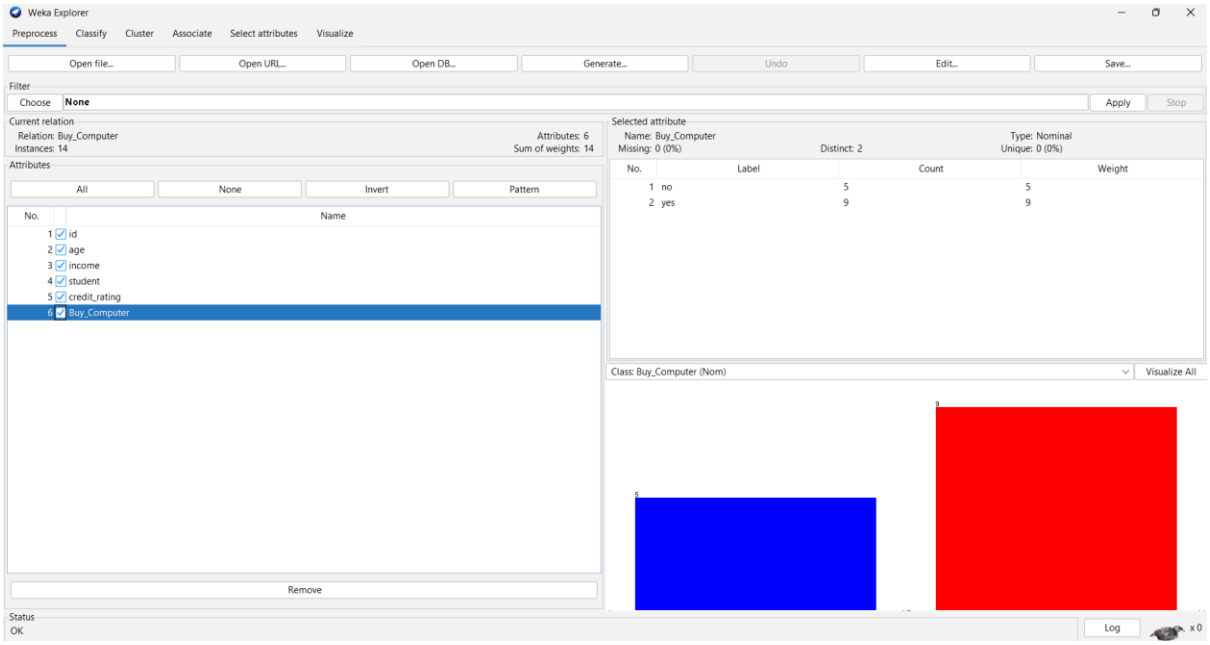
- **Language**: Proprietary (IBM)
- **Interface**: GUI
- **Ease of Use**: Drag-and-drop interface
- **Best For**: Marketing, finance, CRM analytics
- **Features**: Advanced analytics, ML, data prep
- **Limitation**: Commercial license required

---

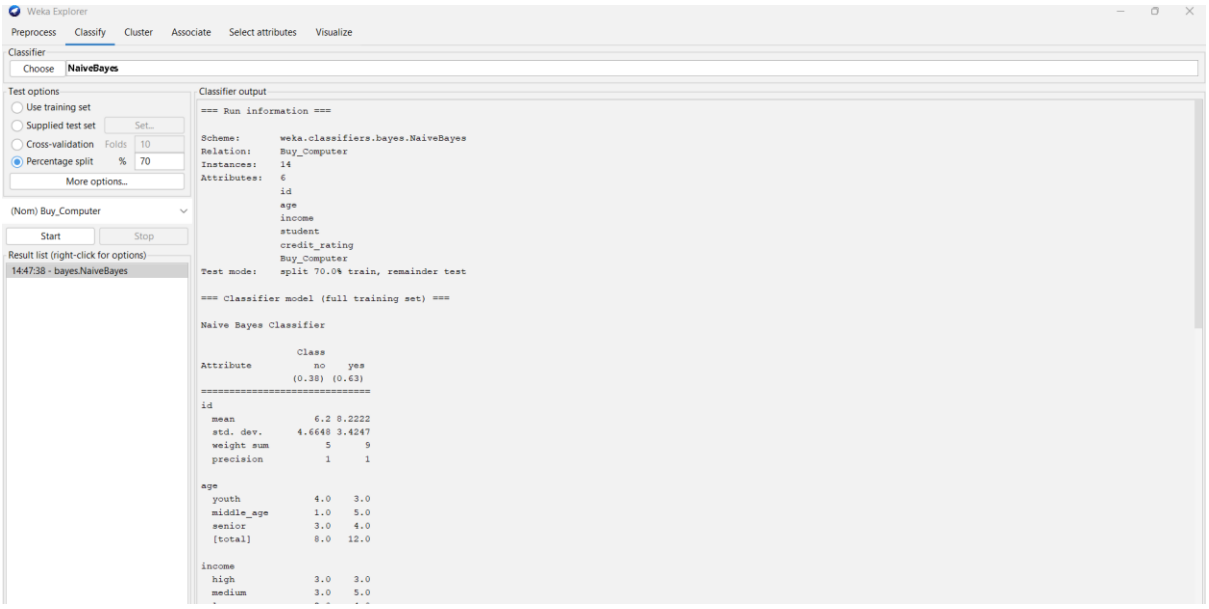
9. H2O.ai

- **Language**: Java, R, Python APIs
- **Interface**: GUI + Code (Flow, Python, R)
- **Ease of Use**: Developer and enterprise friendly
- **Best For**: Scalable machine learning and AI
- **Features**: AutoML, big data support, cloud-native
- **Limitation**: Complex for beginners

Implementation of Classification Algorithm (Naïve Bayes) in WEKA:



Output:



```
income
  high              3.0    3.0
  medium            3.0    5.0
  low               2.0    4.0
  [total]           8.0   12.0

student
  no                5.0    4.0
  yes               2.0    7.0
  [total]           7.0   11.0

credit_rating
  fair              3.0    7.0
  excellent         4.0    4.0
  [total]           7.0   11.0



Time taken to build model: 0 seconds

=== Evaluation on test split ===

Time taken to test model on test split: 0 seconds

=== Summary ===

Correctly Classified Instances           2               50      %
Incorrectly Classified Instances         2               50      %
Kappa statistic                          0
Mean absolute error                      0.4652
Root mean squared error                  0.5224
Relative absolute error                 93.0465 %
Root relative squared error             99.1171 %
Total Number of Instances                4

=== Detailed Accuracy By Class ===
```

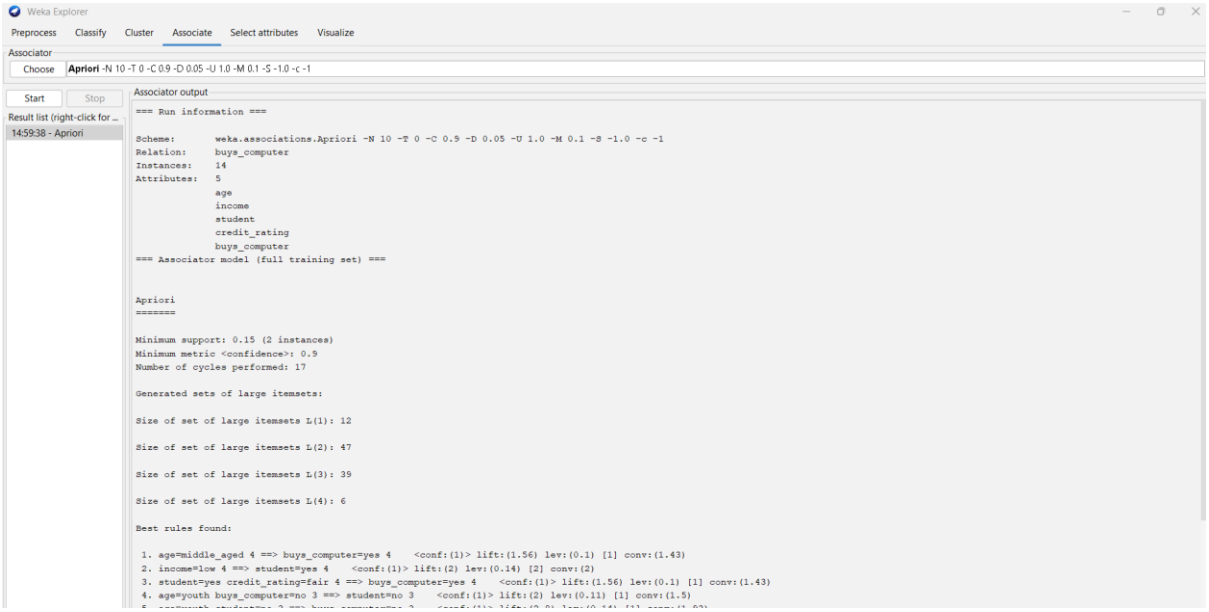
```
=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.000    0.000    ?          0.000    ?          ?        0.750     0.833     no
                 1.000    1.000    0.500      1.000    0.667      ?        0.750     0.833     yes
Weighted Avg.    0.500    0.500    ?          0.500    ?          ?        0.750     0.833

=== Confusion Matrix ===

 a b   <-- classified as
 0 2 | a = no
 0 2 | b = yes
```

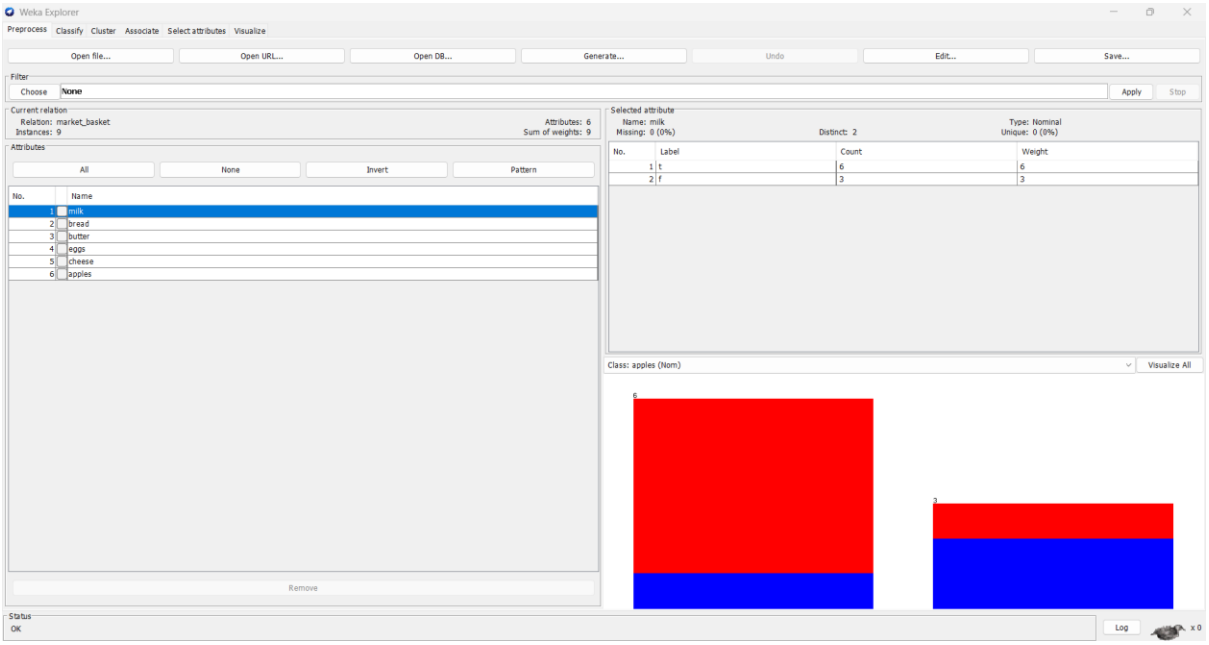
Implementation of Association in WEKA:

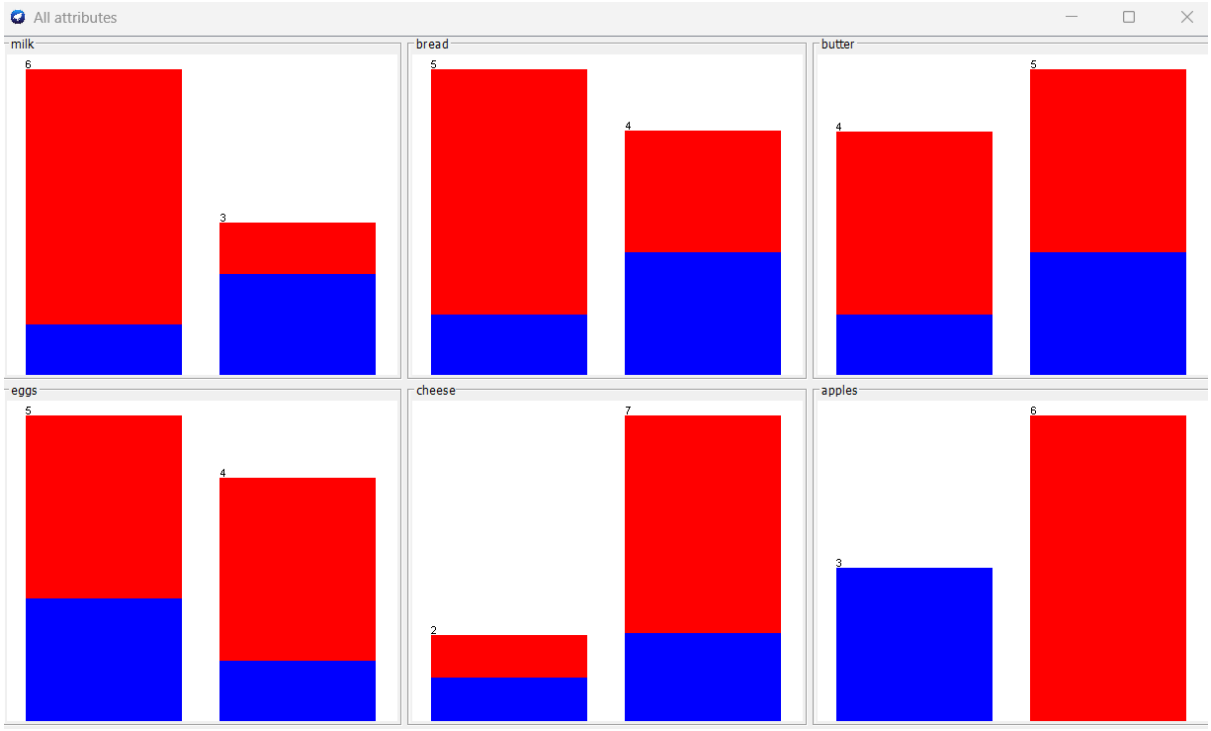


```
Best rules found:

 1. age=middle_aged 4 ==> buys_computer=yes 4    <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
 2. income=low 4 ==> student=yes 4    <conf:(1)> lift:(2) lev:(0.14) [2] conv:(2)
 3. student=yes credit_rating=fair 4 ==> buys_computer=yes 4    <conf:(1)> lift:(1.56) lev:(0.1) [1] conv:(1.43)
 4. age=youth buys_computer=no 3 ==> student=no 3    <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)
 5. age=youth student=no 3 ==> buys_computer=no 3    <conf:(1)> lift:(2.8) lev:(0.14) [1] conv:(1.93)
 6. age=senior buys_computer=yes 3 ==> credit_rating=fair 3    <conf:(1)> lift:(1.75) lev:(0.09) [1] conv:(1.29)
 7. age=senior credit_rating=fair 3 ==> buys_computer=yes 3    <conf:(1)> lift:(1.56) lev:(0.08) [1] conv:(1.07)
 8. income=low buys_computer=yes 3 ==> student=yes 3    <conf:(1)> lift:(2) lev:(0.11) [1] conv:(1.5)
 9. age=youth income=high 2 ==> student=no 2    <conf:(1)> lift:(2) lev:(0.07) [1] conv:(1)
10. income=high buys_computer=no 2 ==> age=youth 2    <conf:(1)> lift:(2.8) lev:(0.09) [1] conv:(1.29)
```

Implementations of Apriori algorithm using WEKA for 9 transactions dataset:




All attributes
milk
6
3
bread
5
4
butter
4
5
eggs
5
4
cheese
2
7
apples
3
6

Output:


Weka Explorer
Preprocess Classify Cluster Associate Select attributes Visualize
Associator
Choose Apriori -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Start Stop
Result list (right-click f...
15:04:33 - Apriori
Associator output
=== Run information ===
Scheme: weka.associations.Apriori -N 10 -T 0 -C 0.9 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Relation: market_basket
Instances: 9
Attributes: 6
milk
bread
butter
eggs
cheese
apples
=== Associator model (full training set) ===
Apriori
=======
Minimum support: 0.35 (3 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 13
Generated sets of large itemsets:
Size of set of large itemsets L(1): 11
Size of set of large itemsets L(2): 23
Size of set of large itemsets L(3): 17
Size of set of large itemsets L(4): 5
Best rules found:
1. bread=t 5 ==> cheese=f 5 <conf:(1)> lift:(1.29) lev:(0.12) [1] conv:(1.11)
2. eggs=f 4 ==> milk=t 4 <conf:(1)> lift:(1.5) lev:(0.15) [1] conv:(1.33)
3. butter=t 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
4. eggs=f 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
Status
OK
Log
x 0

Best rules found:

1. bread=t 5 ==> cheese=f 5 <conf:(1)> lift:(1.29) lev:(0.12) [1] conv:(1.11)
2. eggs=f 4 ==> milk=t 4 <conf:(1)> lift:(1.5) lev:(0.15) [1] conv:(1.33)
3. butter=t 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
4. eggs=f 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
5. milk=t bread=t 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
6. eggs=f cheese=f 4 ==> milk=t 4 <conf:(1)> lift:(1.5) lev:(0.15) [1] conv:(1.33)
7. milk=t eggs=f 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
8. eggs=f 4 ==> milk=t cheese=f 4 <conf:(1)> lift:(1.8) lev:(0.2) [1] conv:(1.78)
9. bread=t apples=f 4 ==> cheese=f 4 <conf:(1)> lift:(1.29) lev:(0.1) [0] conv:(0.89)
10. milk=f 3 ==> eggs=t 3 <conf:(1)> lift:(1.8) lev:(0.15) [1] conv:(1.33)